

Competitive Programming

Section 15: BFS And Unweighted Shortest Paths

Competitive Programming Course

2026-07-05

Table of contents

1	Why graphs now	3
2	Recognition	4
3	Vocabulary	5
3.1	Graph	5
3.2	Directed and undirected edges	5
3.3	Adjacency list	6
4	Queue primer	7
5	The BFS invariant	8
6	Visited vs distance	9
7	Worked example 1: connected components	10
8	Worked example 2: shortest distances	11
9	Grid BFS	12
10	Parent reconstruction	13
11	Tie-breaking paths	14
12	Complexity	15
13	Common wrong approaches	16
13.1	Re-running BFS for every query	16
13.2	Forgetting directedness	16
13.3	Marking visited too late	16
13.4	Using BFS with weighted edges	16
14	Implementation notes	17
14.1	C++	17
14.2	Python	17
15	Local exercises	18
15.1	A. Connected Groups	18
15.2	B. Unweighted Routes	18
15.3	C. Grid Rescue Path	18
16	Submission checklist	19

1 Why graphs now

Phase 1 was mostly about arrays, intervals, and ordering. Phase 2 begins the graph part of competitive programming.

A graph is a way to model states and allowed moves:

```
vertex = a position, object, city, cell, number, state, ...  
edge   = one allowed transition between two vertices
```

Breadth-first search, usually called BFS, is the first graph algorithm you should make automatic. It answers questions like:

```
How many connected groups are there?  
Can I reach this target?  
What is the fewest number of moves?  
Which shortest path should I output?
```

BFS works when every edge has the same cost. In most contest statements that means each move costs 1.

2 Recognition

Look for these signals:

Statement signal	Likely tool
“minimum number of moves” and every move has equal cost	BFS distance
rooms, islands, components, connected groups	BFS/DFS traversal
grid with moves up/down/left/right	grid BFS
“unreachable” should print -1 or NO	BFS reachability
output a shortest path	BFS with parent reconstruction
edge weights are not all equal	not plain BFS; later sections teach Dijkstra

The constraint signal is also important. If n, m are up to $2 * 10^5$, an $O(n+m)$ graph traversal is intended. An adjacency matrix with n^2 memory is not viable at that size.

3 Vocabulary

3.1 Graph

A graph has vertices and edges. We often write:

```
V = number of vertices
E = number of edges
```

In code, the vertices are usually integers.

Contest statements may number vertices from 1 to n, but arrays are easier with 0 to n-1. Convert immediately:

```
cin >> u >> v;
--u;
--v;
```

```
u -= 1;
v -= 1;
```

3.2 Directed and undirected edges

An undirected edge can be used both ways:

```
u -- v
```

Store it twice:

```
adj[u].push_back(v);
adj[v].push_back(u);
```

A directed edge can be used only in one direction:

```
u -> v
```

Store it once:

```
adj[u].push_back(v);
```

3.3 Adjacency list

An adjacency list stores, for every vertex, the vertices you can go to next.

For this graph:

```
1 -- 2 -- 4
|    |
3 -- 5
```

using zero-based vertex names internally, the list is:

```
0: 1, 2
1: 0, 3, 4
2: 0, 4
3: 1
4: 1, 2
```

C++:

```
vector<vector<int>> adj(n);
```

Python:

```
adj = [[] for _ in range(n)]
```

This uses $O(V+E)$ memory. That is the default representation for sparse contest graphs.

4 Queue primer

BFS uses a queue. A queue is first-in, first-out:

```
push 2, push 5, push 8
pop -> 2
pop -> 5
pop -> 8
```

C++:

```
queue<int> q;
q.push(start);
int u = q.front();
q.pop();
```

Python:

```
from collections import deque

q = deque([start])
u = q.popleft()
q.append(v)
```

Do not use `list.pop(0)` in Python for BFS. It shifts the whole list and can turn an $O(V+E)$ traversal into a slow solution.

5 The BFS invariant

BFS explores vertices in increasing distance from the start.

The standard code:

```
dist[start] = 0
push start into queue

while queue is not empty:
    u = pop front
    for every neighbor v of u:
        if v was not visited:
            dist[v] = dist[u] + 1
            push v
```

The key invariant:

```
When a vertex is first discovered, its distance is already the shortest
possible distance from the start.
```

Why? The queue processes layer by layer:

```
distance 0: start
distance 1: neighbors of start
distance 2: neighbors of distance-1 vertices not seen before
distance 3: neighbors of distance-2 vertices not seen before
...
```

ASCII trace:

Graph:

```
1 -- 2 -- 4 -- 6
|       |
3 -- 5 --+
```

Start: 1

```
Layer 0: 1
Layer 1: 2, 3
Layer 2: 4, 5
Layer 3: 6
```

Vertex 6 may be reached as 1-2-4-6 or 1-3-5-4-6, but BFS discovers it from the first layer that can reach it, so the distance is 3.

6 Visited vs distance

For reachability only, a Boolean array is enough:

```
vector<char> seen(n, false);
```

For shortest distances, use an integer array initialized to -1:

```
vector<int> dist(n, -1);  
dist[start] = 0;
```

The value -1 means “not discovered yet”. This avoids maintaining both `seen` and `dist`.

7 Worked example 1: connected components

Suppose the graph is:

```
1 -- 2 -- 3      4 -- 5      6
```

There are three connected components:

```
{1,2,3}, {4,5}, {6}
```

Algorithm:

```
answer = []
for every vertex s:
    if s is unseen:
        run BFS from s
        count how many vertices this BFS visits
        append that count
```

Trace:

```
s = 1: BFS visits 1,2,3 -> size 3
s = 2: already seen
s = 3: already seen
s = 4: BFS visits 4,5 -> size 2
s = 5: already seen
s = 6: BFS visits 6 -> size 1
```

Component sizes are 3,2,1.

This is not a shortest-path problem, but it is still BFS. BFS is also a general “visit everything reachable from here” tool.

8 Worked example 2: shortest distances

Directed graph:

```
1 -> 2 -> 4 -> 5
|           ^
v           |
3 -----+
6 -> 5
```

Start from 1.

Queue and distances:

```
initial:
queue = [1]
dist[1] = 0

pop 1:
discover 2,3
queue = [2,3]
dist[2] = 1, dist[3] = 1

pop 2:
discover 4
queue = [3,4]
dist[4] = 2

pop 3:
4 is already discovered, do nothing
queue = [4]

pop 4:
discover 5
queue = [5]
dist[5] = 3
```

Vertex 6 is unreachable from 1, so $\text{dist}[6] = -1$.

9 Grid BFS

A grid is also a graph.

Each open cell is a vertex. Each allowed move between adjacent cells is an edge.

For a cell (r,c) , the four neighbors are:

```
(r+1,c) down
(r,c-1) left
(r,c+1) right
(r-1,c) up
```

Diagram:

```
      (r-1,c)
        U
(r,c-1) L [r,c] R (r,c+1)
        D
      (r+1,c)
```

Implementation pattern:

```
int dr[4] = {1, 0, 0, -1};
int dc[4] = {0, -1, 1, 0};
char mv[4] = {'D', 'L', 'R', 'U'};
```

```
DIRS = [(1, 0, "D"), (0, -1, "L"), (0, 1, "R"), (-1, 0, "U")]
```

For each neighbor, check:

```
inside grid?
not a wall?
not discovered before?
```

10 Parent reconstruction

Distances tell you how short the path is. Parents tell you the actual path.

When BFS first discovers v from u , record:

```
parent[v] = u
move_used[v] = character used to move from u to v
```

After BFS, walk backward from the goal:

```
goal -> parent[goal] -> parent[parent[goal]] -> ... -> start
```

Collect the move characters on the way and reverse them.

Example:

```
S . . G

parent of cell 2 = cell 1, move R
parent of cell 3 = cell 2, move R
parent of G      = cell 3, move R

backward moves: R, R, R
after reverse:  RRR
```

11 Tie-breaking paths

Many grids have several shortest paths. A normal online judge often accepts any valid shortest path with a special checker. Our local checker compares tokens, so Problem C specifies a unique path:

```
lexicographically smallest shortest path under D < L < R < U
```

To achieve that with BFS, process neighbors in exactly this order:

```
D, L, R, U
```

BFS discovers all distance- k states before distance- $k+1$ states. Within the same distance, this neighbor order makes the first recorded parent produce the smallest move string under the specified order.

The important rule:

```
Set parent only the first time a cell is discovered.  
Do not replace it later with another path of the same length.
```

12 Complexity

For an adjacency-list graph:

```
Time:  $O(V + E)$   
Memory:  $O(V + E)$ 
```

For an $n \times m$ grid:

```
 $V = n*m$   
E is at most about  $4*n*m$   
Time:  $O(n*m)$   
Memory:  $O(n*m)$ 
```

This is why BFS fits grids up to roughly 10^6 cells in C++ and often Python, with careful input and simple arrays.

13 Common wrong approaches

13.1 Re-running BFS for every query

If many queries ask distance from the same source, run BFS once and answer each query in $O(1)$.

```
bad:  q times BFS ->  $O(q*(V+E))$   
good: one BFS    ->  $O(V+E+q)$ 
```

13.2 Forgetting directedness

If the statement says roads are one-way, store only $u \rightarrow v$.

If it says roads are bidirectional, store both directions.

13.3 Marking visited too late

Mark a vertex when you push it into the queue, not when you pop it. Otherwise the same vertex may be pushed many times.

Good:

```
if unseen:  
    mark seen  
    push
```

13.4 Using BFS with weighted edges

Plain BFS minimizes number of edges, not total weight. If one move can cost 5 and another can cost 1, this section's BFS is not enough. Dijkstra is a future section.

14 Implementation notes

14.1 C++

Use:

```
#include <bits/stdc++.h>
using namespace std;

vector<vector<int>> adj(n);
queue<int> q;
vector<int> dist(n, -1);
```

For large graphs, avoid recursive DFS because recursion depth may overflow. BFS is iterative.

For `seen`, `vector<char>` is compact and simple:

```
vector<char> seen(n, false);
```

14.2 Python

Use:

```
from collections import deque
import sys

data = list(map(int, sys.stdin.buffer.read().split()))
```

For grid input:

```
grid = [list(sys.stdin.readline().strip()) for _ in range(n)]
```

Use `deque`, not a list, for queue operations.

15 Local exercises

15.1 A. Connected Groups

This is BFS as a reachability tool.

You must:

```
build an undirected adjacency list
start BFS from every unseen vertex
count the size of each component
sort the sizes
```

15.2 B. Unweighted Routes

This is BFS as a shortest-distance tool.

You must:

```
build a directed adjacency list
run BFS once from s
answer all target queries from dist[]
```

15.3 C. Grid Rescue Path

This is BFS with parent reconstruction.

You must:

```
find S and G
run grid BFS with neighbor order D,L,R,U
record parent cell and move character
reconstruct the path if G was reached
```

16 Submission checklist

Before submitting:

```
Did I convert 1-indexed graph input to 0-indexed arrays?  
Did I store undirected edges in both directions?  
Did I store directed edges in only one direction?  
Did I mark vertices when pushing them?  
Did I use deque/queue, not Python list pop(0)?  
For grids, did I check bounds before indexing?  
For path output, did I reverse the reconstructed moves?
```